OnPay

Any-to-Stable Payment Gateway on Solana

Complete Project Document — Business & Technical

Version 1.0 · April 2026

Solana Frontier Hackathon 2026

Colosseum · Payments & Remittance

Flexible Input, Stable Output — Built in Bali

OnPay

Any-to-Stable Payment Gateway on Solana

Complete Project Document — Business & Technical

Version 1.0 · April 2026
Solana Frontier Hackathon 2026 · Colosseum · Payments & Remittance

---

How to read this document

This document has two audiences and is structured so either can read it from start to finish.

- **If you're a non-technical reader** (merchant, investor, friend, curious tourist): start at the beginning. Every chapter leads with plain-language explanations and real-world analogies. Skip any section labeled "🔧 **Technical Deep-Dive**" — you won't miss anything important.
- **If you're a developer or technical reader**: read everything. The "🔧 **Technical Deep-Dive**" sections contain the implementation details, architecture diagrams, code patterns, and design decisions you'll care about.

There is no jargon tax. If a word like "blockchain" or "token" appears in a non-technical section, it will be explained the first time.

Table of Contents

1. What is OnPay, in plain English
2. The problem we're solving
3. How OnPay works — for a non-crypto person
4. Crypto and money, explained for beginners
5. The user experience — step by step
6. How merchants "top up" or get their money
7. All features — MVP and future
8. Inputs and outputs — what goes in, what comes out
9. How OnPay helps different types of users
10. Business logic — how the business actually runs

11. Revenue model and unit economics
12. Go-to-market strategy
13. How to win the hackathon — strategy & execution
14. Implementation plan — timeline, milestones, budget
15. Budget breakdown
16. The competition & why we're different
17. 🔧 Technical architecture
18. 🔧 Technology stack — what we're building with
19. 🔧 Smart contract and transaction design
20. 🔧 Security, non-custodial guarantees, risk mitigation
21. 🔧 Development workflow and quality standards
22. Risk analysis
23. Success metrics
24. What happens after the hackathon
25. Closing thoughts

---

1. What is OnPay, in plain English

Imagine you walk into a café in Bali holding a phone. You want to pay for a coffee, but instead of handing over cash or tapping a credit card, you open a wallet app on your phone and scan a QR code on the counter. Two seconds later, the café gets paid. No card processing fees, no bank delay, no middleman holding your money.

That's OnPay.

OnPay is a payment system that lets small businesses accept payments from anyone in the world, using any digital currency the customer already has, and always receive a stable, dollar-equivalent amount in their own wallet. It works the same way a card reader does at a store — except **cheaper, faster, and without needing a bank, a card company, or any central company holding the money in between**.

In one sentence: OnPay turns "accept crypto" into a printed sticker on the counter, and turns "pay with crypto" into a one-tap action.

The three promises of OnPay

1. **Flexible input.** The customer can pay with whatever digital money they already hold. Doesn't matter if it's SOL, BONK, USDT, or any of dozens of other coins. They don't need to swap or convert first.
2. **Stable output.** The merchant always receives a stable digital dollar (USDC) — the same as having US dollars in a digital wallet. No exposure to price swings, no volatility, no guessing what the money will be worth tomorrow.
3. **Nobody in the middle.** OnPay's servers generate the QR code and build the payment instructions, but **OnPay never touches the money**. The payment goes directly from the customer's wallet, through a currency swap, into the merchant's wallet — all in one single step that either fully succeeds or fully cancels. There is no "holding period," no "pending settlement," no escrow.

2. The problem we're solving

For merchants (small business owners)

Imagine you run a small café in Canggu, Bali. You serve maybe 80 customers a day, average ticket $4. That's $320 a day in sales, $9,600 a month.

Here's what happens with a regular card reader:

- The customer pays $4.
- Your bank's card processor takes **2.5% = $0.10** per transaction.
- You actually receive $3.90.
- The $3.90 doesn't show up in your bank account for **2–3 business days**.
- Every so often, a customer disputes a charge and you lose the money entirely (chargeback risk).

Your card processor fees alone cost you about $240 a month — that's four days of rent, or a week of groceries. And your cash flow is always delayed.

Now let's add the other pain point: **crypto tourists**. You're in Bali. Every day, 3–5 customers ask "can I pay with crypto?" and you say no. Those are sales you lose to the café next door that figured it out.

For buyers (the crypto-holding customer)

You're a European tourist in Bali. You hold SOL, USDC, maybe some BONK or JUP from a trade. You want to actually spend this money, not swap it back into euros at a shady exchange booth.

Today, your options are:

1. **Use a regular ATM** — pay 2-5% in foreign transaction fees, plus ATM fees, plus withdraw cash you don't really want.
2. **Use an "accept crypto" app** — but each one only accepts one specific coin. You don't have that coin. You'd have to manually swap first, costing time and a fee.
3. **Pretend you don't hold crypto** — frustrating, and you lose the main reason you carry it.

None of those options are good. You give up and pay with a card, accepting the fee.

For the Solana blockchain ecosystem

Solana is a very fast, very cheap blockchain. It can process thousands of payments per second for fractions of a cent. It was practically designed to be the rail for real-world commerce.

But today, nobody has shipped a simple, merchant-first, non-custodial payment product that actually uses this capability. The tools exist (Solana Pay, Jupiter, SPL tokens), but nobody has glued them together into a product a non-technical café owner can use in 60 seconds.

That's the opportunity.

---

3. How OnPay works — for a non-crypto person

Let's walk through a real purchase, end to end, with zero jargon.

The café owner's side (one-time setup)

1. **The café owner, Kadek, visits** `onpay.app` on her phone or laptop.
2. **She taps "Start accepting payments"** and a pop-up asks her to connect a wallet.
3. **She has Phantom** (a free wallet app, like a digital version of her physical wallet) installed. She taps "Connect Phantom."
4. **Phantom asks her to confirm.** She taps yes.
5. **She's done.** OnPay now knows she's a merchant. She can create invoices. Total setup time: under 60 seconds.

> *What is a "wallet" here? Think of it as the digital equivalent of her physical wallet. It's an app on her phone that holds her money and her identity. It's controlled only by her — OnPay cannot access it, touch it, or freeze it. The wallet is her identity for OnPay; she didn't create a password or sign up with email. The wallet IS the account.*

Creating an invoice

A customer walks up to order a $4 latte. Kadek:

1. **Opens her OnPay dashboard** on her phone (or a tablet on the counter).
2. **Taps "New payment."**
3. **Types "4" for the amount** (in US dollars, or in Indonesian rupiah — she can pick).
4. **Taps "Generate QR."**
5. **A big QR code appears on her screen.** She holds it up to the customer.

The customer's side

The customer, Alex, holds a mix of crypto in his phone wallet. He:

1. **Opens his wallet app** (Phantom or Backpack).
2. **Taps "Scan."**

3. **Points the camera at Kadek's QR code.**
4. **His wallet shows a screen:**
   > "Pay $4.00 to this merchant.
   > You can pay with: SOL, USDC, BONK, JUP... (list of the coins he holds)
   > Pick one."
5. **Alex picks BONK** (he has a bunch from a trade and wants to get rid of it).
6. **His wallet calculates:** "That'll be approximately 52,000 BONK. The merchant will receive exactly $4.00 worth of USDC. Max price swing: 0.5%. Approve?"
7. **Alex taps "Approve."**

What happens in the next 2 seconds

In the background, the following happens automatically, as one single atomic action — meaning either everything succeeds or nothing happens at all:

1. Alex's BONK is sent to a swap service (Jupiter) which automatically trades it for USDC at the current market rate.
2. That USDC is immediately sent to Kadek's wallet address.

Two seconds later:

- **Kadek's dashboard** shows "Paid ✓ $4.00 received." She hands Alex his latte.
- **Alex's wallet** shows "Payment complete, BONK sent." He walks away with his coffee.

Nobody held the money in between. Not OnPay, not Jupiter, not any bank. It went straight from Alex's wallet to Kadek's wallet in one step. The swap happened inside the same transaction as the transfer.

The "invisible" parts

What Alex never had to do:
- Understand blockchain
- Swap his BONK to the merchant's preferred coin manually
- Pay a separate swap fee on top of a payment fee
- Wait for multiple confirmations

What Kadek never had to do:
- Install a card reader
- Pay monthly hardware fees
- Wait 3 days for settlement
- Deal with chargebacks
- Hold any volatile coin — she received USDC, which is always worth $1

That's the OnPay experience: the complexity exists, but users never see it.

---

4. Crypto and money, explained for beginners

Skip this chapter if you already know what a blockchain and a stablecoin are.

What is "digital money"?

You're probably already familiar with digital money. Your bank account isn't a pile of cash in a vault — it's a number on a computer at your bank. When you swipe your card, banks on both sides update their computers to say "Alice has $4 less, Bob's shop has $4 more."

The problem: banks are slow, expensive, and centralized. Every transaction has to go through them, they charge fees, they're open only during certain hours, and they can decide to freeze your account.

What is a blockchain?

A blockchain is a different way to store and move digital money, without a central bank in charge.

Instead of one bank's computer holding the record of who owns what, thousands of computers around the world hold the same record and constantly agree on it. When you want to send money, you broadcast a message to this network: "I, Alice, want to send $4 to Bob." The network checks that Alice actually has $4, and updates the shared record.

Because there's no central authority:
- Nobody can freeze your account (not a bank, not a government, not even the people who built the blockchain).
- Transfers can happen 24/7, worldwide, in seconds.
- Fees are much lower because there's no company extracting a margin.

The trade-off: you're responsible for your own "wallet." If you lose your wallet's key (like losing a physical safe's combination), you lose access forever. No customer service can reset it.

What is Solana?

Solana is one specific blockchain — the one OnPay is built on.

We picked Solana because it's:
- **Fast.** Transactions confirm in about 1 second, fast enough for in-person payments.
- **Cheap.** Fees are fractions of a cent, so a $0.50 transaction is economically viable.
- **Has good tools.** There's a standard for QR payments (Solana Pay) and a great swap service (Jupiter).

Other blockchains like Ethereum are either too slow or too expensive for real-world payments.

What are "tokens" and "SPL tokens"?

On Solana, the native currency is called SOL. But the blockchain can also hold other kinds of digital money — "tokens" — that represent different things. An SPL token (Solana Program Library token) is any custom token on the Solana blockchain.

Examples of SPL tokens on Solana:
- **USDC** — a "stablecoin" pegged to the US dollar. 1 USDC = $1. Always.
- **USDT** — another stablecoin pegged to the US dollar.
- **BONK** — a popular meme-coin on Solana.
- **JUP** — the token of the Jupiter exchange.

Merchants prefer USDC because it's stable — always worth $1. Buyers might hold any mix of the above.

What is a "stablecoin"?

A stablecoin is a kind of digital money designed to always be worth exactly one US dollar (or sometimes one euro, one yen, etc.). Companies like Circle (USDC) and Tether (USDT) back each stablecoin with a real dollar held in reserve.

Why merchants want stablecoins:
- Zero price volatility. If you accept $100 in USDC today, it's worth $100 tomorrow.
- Same units as fiat accounting. Merchants can price in dollars and track revenue without conversion math.

What is a "swap" and what is Jupiter?

A swap is exchanging one kind of digital money for another, like converting euros to dollars at a currency exchange booth.

Jupiter is a service on Solana that finds the best swap rate across all available exchanges. If you want to convert BONK to USDC, Jupiter checks 20+ exchanges and picks the best route. It's what OnPay uses under the hood to convert whatever token the buyer pays with into the USDC the merchant wants.

What is a "non-custodial" system?

This is the most important concept for OnPay.

Custodial means a company holds your money on your behalf. Your bank is custodial — the dollars in your bank account are technically the bank's, and they owe them to you. If the bank fails, you might lose the money. Most crypto exchanges (Coinbase, Binance) are also custodial.

Non-custodial means nobody holds your money except you. The money lives in your own wallet, controlled by your own private key. No company can freeze, seize, or lose your funds — because no

company has them in the first place.

OnPay is non-custodial. When a buyer pays a merchant through OnPay:
- OnPay's servers generate a transaction *instruction* (a recipe: "swap this, transfer that").
- The buyer signs the instruction with their wallet.
- The transaction executes directly on the Solana blockchain.
- OnPay never possesses the money. Not for a millisecond.

If OnPay's servers disappeared tomorrow, every past payment is still safely in its merchant's wallet. This is not marketing — it's a mathematical property of how the system is built.

What is a "QR code payment"?

A QR code is a pattern of black and white squares that encodes information. When your phone's camera reads it, it can extract that information — a URL, a phone number, or in our case, payment instructions.

On Solana, there's a standard called **Solana Pay** that defines exactly how payment QR codes should work. Every major Solana wallet (Phantom, Backpack, Solflare) can scan a Solana Pay QR and handle the payment automatically.

OnPay uses the most flexible version of Solana Pay called **"Transaction Request"** — this means the QR doesn't just encode a fixed payment, it tells the buyer's wallet to ask OnPay's server for a custom transaction. That's how we can dynamically build the any-to-stable swap based on whatever token the buyer decides to pay with.

---

5. The user experience — step by step

For the merchant (Kadek, café owner)

Day 1 — Onboarding (under 60 seconds)

1. Visits `onpay.app` on her phone
2. Taps "Start accepting payments"
3. Connects Phantom wallet (one tap)
4. Enters business name: "Kopi Canggu"
5. Default settlement coin: USDC (pre-selected — she doesn't need to change it)
6. Done. Dashboard loads.

Daily usage — creating payments

Every time a customer orders:
1. Opens OnPay dashboard on her phone or tablet
2. Taps "New payment"
3. Types the amount: "4" (in USD) or "60000" (in IDR — for display only)
4. Optional: adds label "Iced Latte x2"
5. Taps "Generate QR"
6. Holds the phone up to the customer, who scans it
7. Receives instant notification: "Paid ✓ $4.00 USDC received"
8. Hands over the coffee

Weekly/monthly usage — reviewing earnings

1. Opens dashboard
2. Sees cards: "Today: $87 (22 txns)", "This week: $612 (148 txns)", "This month: $2,340 (587 txns)"
3. Scrolls transaction history
4. Taps any transaction to see details (time, amount, buyer wallet truncated, on-chain link)
5. (Phase 2) Taps "Cash out to IDR" to send USDC to her local bank via an off-ramp partner

For the buyer (Alex, crypto-holding tourist)

First time paying with OnPay

1. Walks up to the café counter

2. Sees the QR on Kadek's phone
3. Opens Phantom on his phone, taps "Scan"
4. Points camera at the QR
5. Wallet shows: "Pay $4.00 to Kopi Canggu. Pick payment token:"
6. Taps "BONK" (has a bunch, wants to spend it)
7. Wallet shows: "52,000 BONK ≈ $4.00 USDC. Slippage 0.3%. Approve?"
8. Taps "Approve"
9. Signs (biometric or PIN)
10. Two seconds later: "Payment successful. Receipt saved to your wallet."
11. Walks away with coffee

Second time — even faster

1. Scans QR
2. Picks token
3. Approves
4. Done

For the developer (Priya, building a Shopify store)

(Phase 2, post-hackathon — shown here to illustrate the long-term vision)

1. Installs the "OnPay" plugin from the Shopify app store
2. Connects her Solana wallet in the plugin settings
3. Publishes her store
4. Her checkout page now shows "Pay with crypto" as an option
5. When a customer picks it, OnPay handles the full flow — swap, transfer, confirmation
6. Priya receives USDC directly, with a webhook notification to her store admin

---

6. How merchants "top up" or get their money

This is a common question for non-crypto people: "How do I actually get money out of this thing?"

Inbound (receiving payments)

Merchants don't need to "top up" anything to *receive* payments. They just need:
1. A Solana wallet (free, takes 30 seconds to install Phantom)
2. A merchant account on OnPay (free, auto-created when they connect their wallet)

Every payment lands in their wallet instantly. They don't pre-fund anything.

Outbound (cashing out to local currency)

This is where the question gets important. USDC in a wallet is like having US dollars — technically. But for a Bali café owner, what actually matters is whether she can buy groceries and pay rent in Indonesian rupiah (IDR).

Option 1: Keep it in USDC (no action needed)

If Kadek is happy holding USD-equivalent funds, she does nothing. Her USDC stays in her wallet. She can use it to pay any other OnPay merchant. She can send it to family abroad instantly (international remittance is a Phase 2 use case). She can swap it to other tokens if she wants.

Option 2: Cash out via an exchange (available today)

Kadek can:
1. Send her USDC to a crypto exchange that operates in Indonesia (Pintu, Tokocrypto, Indodax)
2. Sell the USDC for IDR on the exchange
3. Withdraw the IDR to her local bank account

This takes about 30 minutes end-to-end and typically costs 0.3–0.8% in exchange fees.

Option 3: OnPay's integrated fiat off-ramp (Phase 2 — coming post-hackathon)

This is the killer feature for merchant adoption in Indonesia. In Phase 2, OnPay will partner with a local licensed PSP (payment service provider) that specializes in crypto-to-fiat conversion. Flow:
1. Kadek opens OnPay dashboard
2. Taps "Cash out $500 USDC to my bank"

3. Enters her BCA/Mandiri account number (saved for next time)
4. Confirms
5. Within 24 hours, IDR lands in her bank account
6. Fee: ~0.5% spread

Compared to the current 2.5% card processor fee + 2–3 day settlement, that's **5x cheaper and 3x faster**.

Buyer side — how does a buyer "top up"?

The buyer needs to have crypto in their wallet before paying. Options:

1. **Already holds crypto** (the tourist persona — common in Bali)
2. **Buys crypto in their home country** via a centralized exchange, then transfers to their Solana wallet
3. **Uses the wallet's built-in "Buy" feature** — Phantom and Backpack have built-in on-ramps (MoonPay, Ramp) that let you buy SOL or USDC directly with a credit card
4. **(Phase 2, optional)** OnPay could integrate a card-to-crypto on-ramp directly into the payment flow, so a buyer with no crypto could pay with a card and the merchant would still receive USDC. This is a stretch goal — the core product assumes buyers arrive with crypto already.

---

7. All features — MVP and future

Phase 1 — Hackathon MVP (built by May 11, 2026)

Merchant features

- **Wallet-based authentication.** Connect Phantom, Backpack, or Solflare. The wallet public key IS the merchant ID. No passwords, no email, no KYC.
- **Merchant profile.** Business name, preferred settlement token (default USDC), optional display logo.
- **Invoice creation.** Enter amount (USD primary, IDR display), optional label and memo.
- **QR code generation.** Solana Pay Transaction Request URL, rendered as a fullscreen-friendly QR.
- **Real-time payment notifications.** Dashboard updates within 1–2 seconds of payment confirmation.
- **Transaction history.** Paginated list with status, amount, time, truncated buyer wallet, on-chain link.
- **Analytics summary.** Today / this week / this month revenue and transaction count cards.
- **Devnet/mainnet toggle.** Safe testing mode for development; production mode for real payments.
- **Accessibility-first UI.** Keyboard navigable, screen-reader friendly, WCAG AA color contrast.
- **Internationalization.** English + Bahasa Indonesia.

Buyer features

- **One-scan payment.** Scan QR → pick token → sign → done.
- **Any SPL token input.** Jupiter routes any token with a liquid route to USDC.
- **Transparent pricing preview.** Wallet shows exact input amount, output amount, slippage before signing.
- **Receipt.** Buyers see a confirmation screen with tx hash and merchant info.
- **No account required.** Buyers just use their existing wallet. No OnPay signup.

Backend / infrastructure

- **Solana Pay Transaction Request endpoint.** Dynamically builds transactions per buyer.
- **Jupiter v6 integration.** Quote and swap-instructions endpoints.
- **Atomic transaction composition.** Swap + transfer in one signed transaction.
- **Reference key system.** Unique cryptographic reference per invoice for tracking.
- **Rate limiting.** Per-IP and per-reference limits to prevent abuse.
- **Slippage cap.** Configurable maximum slippage (default 1%).
- **Supabase storage.** Merchants, invoices, payments tables with Row-Level Security.
- **Vercel deployment.** Next.js frontend + API routes in one bundle.

- **CI/CD.** GitHub Actions running typecheck, lint, tests on every push.

Phase 2 — Post-hackathon (months 1–6)

Fiat rails

- **IDR off-ramp** via licensed Indonesian PSP (Pintu, Reku, or direct partnership). Merchant receives IDR in BCA/Mandiri within 24 hours.
- **Multi-country off-ramps.** USD (ACH), EUR (SEPA), PHP, THB, VND (Southeast Asia tourism corridor).
- **Card-to-crypto on-ramp.** Optional: buyer with no crypto pays by card, merchant still receives USDC.

Subscription & recurring payments

- **Anchor program** for permissioned debit — merchant can charge a subscribed wallet monthly without the buyer signing each time.
- **Subscription management dashboard** for merchants (active subs, churn, MRR).
- **Buyer-side subscription manager** in wallet.

E-commerce integrations

- **TypeScript SDK** (`@onpay/sdk`) published to npm. Drop-in helpers for any Node/React/Next project.
- **Shopify plugin.** Listed on the Shopify App Store.
- **WooCommerce plugin.** WordPress-compatible.
- **Wix plugin.** For the no-code crowd.
- **Stripe-compatible webhook API.** So existing e-commerce backends that already handle Stripe webhooks can plug in OnPay with minimal changes.

Growth features

- **Loyalty cNFTs.** Automatic compressed NFT minting after a merchant-configured number of payments (e.g., "10th coffee free"). Compressed NFTs cost cents per 1000 mints on Solana.
- **Referral system.** Merchants earn fees from customers they refer to other OnPay merchants.
- **Multi-merchant routing.** Marketplace-style split payments (90% to vendor, 10% to platform).

Advanced merchant tools

- **Multi-device dashboard.** Same merchant account across phone, tablet, cashier terminal.
- **Staff permissions.** Merchant owner can delegate "create invoice" rights to staff without exposing the wallet key.
- **Refunds.** Merchant can issue a partial or full refund (non-custodial: uses the merchant's own wallet to send USDC back).
- **Export.** CSV export of transactions for accounting.
- **Native mobile merchant app.** iOS + Android, optimized for cashier usage.

8. Inputs and outputs — what goes in, what comes out

Party	Input (what they provide)	Output (what they receive)
Buyer	Any supported SPL token (SOL, USDC, USDT, BONK, JUP, and 50+ others with a Jupiter route) + one wallet signature	Confirmed payment, on-chain receipt in their wallet
Merchant	Wallet connection + invoice amount (USD)	USDC directly in their wallet within 2 seconds of buyer signing
OnPay backend	Invoice reference key + buyer's public key (from the scanned QR)	A serialized, unsigned atomic transaction ready for the buyer to sign
Jupiter	Input token + desired output (USDC) + amount	A quote + swap instructions for the best available route
Solana blockchain	A signed transaction containing the swap and transfer instructions	Execution of the transaction + a confirmed tx hash

9. How OnPay helps different types of users

The Bali café owner (Kadek)

Before OnPay: Loses 2.5% of every card sale, waits 3 days for settlement, turns away crypto-paying tourists.
With OnPay: Pays 0% (MVP) or 0.2–0.5% (production). Receives USDC instantly. Accepts payment from tourists who hold any crypto. **Net benefit: ~$200/month in saved fees, plus recovered tourist revenue, plus better cash flow.**

The crypto tourist (Alex)

Before OnPay: Can't spend his crypto at local businesses. Pays high ATM fees to get cash.
With OnPay: Pays with whatever token he holds, one-tap, at dozens of local merchants. **Net benefit: zero conversion friction, no ATM fees, real-world utility for his crypto.**

The e-commerce store owner (Priya)

Before OnPay (Phase 2): Wants to accept crypto but existing options are custodial, slow to integrate, or require her to hold volatile assets.
With OnPay: Installs a Shopify plugin, receives USDC on every crypto checkout, no custody risk, no new accounting flow. **Net benefit: new crypto revenue channel with zero operational overhead.**

The Southeast Asian tourism corridor

Before OnPay: Fragmented payment infrastructure across countries (IDR, THB, VND, PHP). Each with its own fees and delays. International tourists struggle to pay locals.
With OnPay: A single payment rail works from Bali to Bangkok to Hanoi to Manila. Merchants in every country receive USDC; cashing out to local currency is country-specific but the merchant-facing product is identical. **Net benefit: a real cross-border payment layer for SE Asia.**

The Solana ecosystem

Before OnPay: Solana's real-world-payment story is told in press releases, not products. Solana Pay exists but isn't composed with Jupiter into a merchant-first product.

With OnPay: A reference implementation of non-custodial any-to-stable payments exists, open-source, ready to fork. **Net benefit: proof that Solana is the payment rail for the real economy, plus composable primitives for future builders.**

---

10. Business logic — how the business actually runs

Day in the life of a payment, business-wise

1. **Kadek (merchant) creates invoice for $4.**
   - OnPay generates a unique reference key (32 random bytes).
   - Stores invoice in Supabase: `{merchant: kadek_wallet, amount_usd: 4, reference: 0x...}`.
   - Returns a Solana Pay Transaction Request URL encoding that reference.
   - Renders the URL as a QR code on Kadek's screen.

2. **Alex (buyer) scans the QR.**
   - His wallet sees the URL and hits `GET /api/tx/[reference]` to get label + icon.
   - Displays "Kopi Canggu, $4.00" — buyer's first confirmation screen.
   - Buyer picks BONK as input token.
   - Wallet hits `POST /api/tx/[reference]` with Alex's public key and selected input token.

3. **OnPay backend builds the transaction.**
   - Loads the invoice from Supabase.
   - Calls Jupiter v6 `/quote` endpoint: "Best route for X BONK → $4.00 USDC?"
   - Calls Jupiter v6 `/swap-instructions` to get the actual instructions (not a pre-built transaction, just the instructions).
   - Composes a new transaction:
     - Instruction 1..N: Jupiter swap setup (token accounts, route accounts)
     - Instruction N+1: the actual swap
     - Instruction N+2: an SPL Token transfer of the output USDC to Kadek's wallet
     - Instruction N+3: (optional) a memo with the reference key for on-chain indexing
     - (Production only) an additional SPL transfer skimming 0.2–0.5% to OnPay's treasury PDA
     - Serializes the transaction as a versioned tx (v0) with Address Lookup Tables if needed to fit in 1232 bytes.
     - Returns it to Alex's wallet.

4. **Alex reviews and signs.**
   - Wallet displays the full impact: "Spend 52,000 BONK → Kopi Canggu receives 4.00 USDC. Slippage 0.3%."
   - Alex approves with a biometric.
   - Wallet broadcasts the signed transaction to Solana.

5. **Solana executes atomically.**
   - Jupiter swap runs: BONK → USDC (inside the transaction).
   - SPL transfer runs: USDC → Kadek's wallet (inside the same transaction).
   - Either both succeed, or neither does. No partial state.
   - Confirmed in about 1–2 seconds.
6. **OnPay updates state.**
   - A background worker (or webhook from Helius RPC) detects the confirmed transaction for this reference.
   - Updates Supabase: `payments.status = 'paid', tx_hash = 0x...`
   - Pushes a Supabase Realtime event to Kadek's dashboard.
   - Kadek's screen shows "Paid ✓".

What OnPay charges (production, post-hackathon)

OnPay's revenue comes from three streams:

1. **Protocol fee on payments (primary).** 0.2–0.5% of transaction volume. Collected by adding an extra SPL transfer inside the same atomic transaction that routes a small portion of the swap output to OnPay's treasury PDA. This is still non-custodial: the fee is taken inside the buyer-signed transaction, not held by OnPay off-chain. Free tier for merchants doing under $1k/month.
2. **Fiat off-ramp spread (Phase 2).** ~0.5% spread on USDC → IDR / USD / EUR conversions through partner PSPs. Merchants choose whether to use this or self-cash-out via a crypto exchange.
3. **Premium SDK/plugin tier (Phase 2).** Flat monthly fee ($49–$199) for e-commerce integrations with advanced features: multi-store, custom branding, priority support, advanced analytics. Base SDK is free and open-source.

Unit economics (modeled for a typical merchant)

Scenario: Bali café doing $10k/month in card volume, migrated to OnPay.

Metric	Card processor	OnPay
Volume	$10,000/mo	$10,000/mo
Fee %	2.5%	0.3%
Fee $	$250	$30
Settlement delay	2–3 days	instant
Chargeback risk	0.5% avg	0% (payments are final)
Merchant savings	—	**$220/month**

OnPay revenue per merchant: $30/month avg (at 0.3% fee).
OnPay cost per merchant: ~$5/month (infrastructure, RPC, Supabase).
Gross margin per merchant: ~83%.
Target: 100 merchants in 6 months → ~$3k/mo MRR, ~$36k ARR.
Target: 1,000 merchants in 18 months → ~$30k/mo MRR, ~$360k ARR.

These are conservative numbers on a single market (Bali). Phase 2 SDK/plugin revenue can multiply this 5–10x with minimal marginal cost.

---

11. Revenue model and unit economics

(Detailed breakdown for investors / accelerator reviewers.)

Revenue streams

1. Transaction fees (primary, recurring)

- **Rate:** 0.2% standard, 0.5% on low-liquidity token swaps (covers Jupiter slippage risk).
- **Collection mechanism:** atomic in-transaction skim to treasury PDA. Non-custodial.
- **Free tier:** merchants processing under $1k/month pay 0%.
- **Gross margin:** ~85% (infrastructure + RPC costs are the only variable).

2. Fiat off-ramp spread (Phase 2)

- **Rate:** 0.5% on USDC → IDR and 0.3% on USDC → USD/EUR.
- **Mechanism:** partnership with a licensed PSP that handles the bank transfer; OnPay takes a revenue share.
- **Gross margin:** ~60% (PSP takes a cut).

3. SDK/plugin subscriptions (Phase 2)

- **Tiers:** Free (open-source base), Pro ($49/mo, up to $50k monthly volume), Business ($199/mo, unlimited volume + priority support + custom branding).
- **Gross margin:** ~95% (pure software).

Customer acquisition cost (CAC)

- **Bali MVP cohort:** Door-to-door onboarding. CAC ≈ $15/merchant (3 hours of founder time + a physical QR sticker + coffee with the owner).
- **Phase 2 SDK/plugin cohort:** App store listings + content marketing + founder-led outreach. Target CAC ≈ $100 per plugin subscriber.

LTV

- **Typical merchant lifetime:** 24 months (conservative; SME churn is real).
- **Average transaction fee revenue:** $30/month → **$720 LTV per Bali merchant.**
- **LTV / CAC:** 720 / 15 = **48x.** Extremely healthy.

Break-even

- **Fixed costs (self-funded):** ~$200/month (Vercel Pro, Supabase, Helius, domains). Zero payroll while solo.
- **Break-even merchant count:** ~7 merchants at $30/mo ARPU.
- **Break-even timeline:** end of Month 2 post-launch.

---

12. Go-to-market strategy

Phase 1 — Bali proof of concept (Months 1–3 post-hackathon)

Target: 50 merchants in Canggu, Seminyak, Ubud.

Tactics:
- **Physical door-to-door onboarding.** Wira walks into cafés, explains OnPay in 2 minutes, helps the owner connect a wallet on the spot. Leaves a printed QR sticker on the counter. This scales surprisingly well for the first 100 — it's the founder's highest-leverage activity.
- **Free onboarding kits.** Laminated QR stickers, countertop acrylic stand, "we accept crypto here" English + Bahasa signage. Total kit cost ~$3; handed out free.
- **Zero fees for first 3 months.** Remove all friction. Let merchants get comfortable seeing payments come in before charging anything.
- **Reference customer program.** First 10 merchants get lifetime fee discount in exchange for testimonials, case studies, and referrals.
- **Solana Bali Builders community.** Already plugged in. Leverage co-founder intros, mentor network, community events for early merchant intros and buyer-side awareness.

Success metric: 50 active merchants processing total $50k/month in volume by end of Month 3.

Phase 2 — Regional expansion (Months 4–9)

Target: 300 merchants across Bali + Jakarta + Bangkok.

Tactics:
- Hire first operator (part-time, commission-based) in Jakarta to replicate the Bali playbook.
- Launch Indonesian Rupiah off-ramp with PSP partner (signed LOI required by Month 3).
- Start content marketing: YouTube videos walking merchants through setup, Bahasa + English.
- Partner with Solana Foundation for co-marketing (they love real-world adoption stories).
- Launch TypeScript SDK publicly; begin e-commerce plugin development.

Success metric: $250k/month volume by Month 9.

Phase 3 — Developer channel + SEA scale (Months 10–18)

Target: 1,000 merchants + first 100 SDK/plugin users.

Tactics:
- Ship Shopify + WooCommerce plugins to their respective app stores.
- Launch developer docs site with quickstart, API reference, tutorials.

- Hackathon sponsorships and bounty programs for developer acquisition.
- Hire second operator in Ho Chi Minh City or Bangkok.
- Expand off-ramp to THB, PHP, VND.

Success metric: $2M/month volume, 100 paying SDK subscribers, first institutional interest.

Phase 4 — Global (Month 18+)

- European expansion via the SEPA rail (existing PSP partners).
- LATAM expansion (Mexico, Brazil, Argentina — all high-crypto-adoption markets).
- B2B sales motion for mid-market e-commerce.
- Fundraise a seed round if growth rate justifies it.

---

13. How to win the hackathon — strategy & execution

The Colosseum Solana Frontier Hackathon judges on six explicit criteria. Here's how OnPay addresses each, and how we'll actually win (not just place).

The six criteria

1. **Functionality / code quality**
2. **Potential impact / TAM**
3. **Novelty**
4. **UX leveraging Solana's performance**
5. **Open-source / composability with Solana primitives**
6. **Business plan viability**

How OnPay scores on each

1. Functionality / code quality

- **100% TypeScript, strict mode, zero `any` types.** Judges will read the code.
- **JSDoc on every exported function.** Self-documenting.
- **Tests:** unit tests for transaction composition + Playwright end-to-end test for the full payment flow.
- **Clean commit history** with conventional commit messages. No "WIP" commits.
- **Security-hardened:** CSP headers, Supabase RLS policies, validated inputs, rate limiting, pinned dependencies, slippage caps.
- **Accessibility:** Lighthouse accessibility ≥95, keyboard navigable, screen-reader-tested.

2. Potential impact / TAM

- **Global payments market:** $100B+ revenue pool (Visa + Mastercard alone).
- **Indonesian UMKM segment:** 60M+ small businesses, most currently card-only or cash-only.
- **Southeast Asia tourism corridor:** 100M+ international tourist arrivals/year, many crypto-holding.
- **Clear expansion path:** Bali → Indonesia → SE Asia → Global, each step validated before the next.
- The pitch frames this as infrastructure, not a single-market play.

3. Novelty

- **First non-custodial any-to-stable merchant gateway** composing Solana Pay + Jupiter in one atomic transaction.
- Existing products either force token alignment (Solana Pay native) or take custody (Helio, Stripe Crypto).
- The "buyer picks input token, merchant always gets USDC, one signature, nobody touches funds" framing is novel and memorable.

4. UX leveraging Solana's performance

- **Sub-2-second settlement** — only possible on Solana.
- **Sub-cent fees** — makes $0.50 coffee purchases economically viable.
- **One signature** — not the multi-step "swap then pay" flow on every other payment app.
- Judges explicitly score "how well does this utilize Solana's performance" — OnPay is impossible on any other chain and we'll say so.

5. Open-source / composability

- **MIT license.** Permissive.
- **Publicly readable repo.** Not just "open-sourced" — actively structured for forking.
- **Composes with three Solana primitives:** Solana Pay, Jupiter, SPL Token program.
- **TypeScript SDK (stretch):** if shipped by submission day, becomes a primitive that other builders can compose with.
- Judges love infrastructure plays that make other people's projects easier.

6. Business plan viability

- **Concrete go-to-market:** physical Bali rollout, not a landing page.
- **Unit economics modeled:** LTV/CAC 48x, break-even at 7 merchants.
- **Personal founder advantage:** Wira lives in Bali, has Solana Bali Builders + mentor pass already secured.
- **Revenue-model credibility:** protocol fee is collected atomically, non-custodially, which is a provably-honest alternative to custodial processors.

Winning moves (beyond "just build it well")

Move 1: Nail the demo video

Most hackathon entries lose because the demo is a slide deck or a code walkthrough. Ours is **a real Bali café, a real phone, a real payment in BONK, and a real USDC arrival**. Judges watch hundreds of videos — the only way to stand out is to make them feel the moment of real-world adoption.

Move 2: Ship on mainnet, not just devnet

Devnet is default. Mainnet is signal. We will do ONE real mainnet payment end-to-end during week 4, document it with a transaction hash, and put that hash in the demo video. "This is a real payment on mainnet-beta. Here's the tx hash" is a credibility bomb.

Move 3: Publish the SDK by submission day

Even a rough `@onpay/sdk` v0.1 on npm — with one exported function that generates a Solana Pay Transaction Request URL — is a massive composability signal. Costs one day of engineering. Scores points on criteria 5 and 6 simultaneously.

Move 4: Tell the Bali story

The written form answers and the pitch video both anchor on the **first-person Bali café story**. Judges read hundreds of entries that all sound like "we're building infrastructure for the blockchain economy." We're building for Kadek, who I watched turn away a tourist last Tuesday. That memory sticks.

Move 5: Beat every non-critical deadline by 24 hours

- Colosseum individual registration: submit by May 3 (deadline May 4).
- Project submission: submit by May 10 (deadline May 11).
- Leaves a full day's buffer for force-majeure issues. Judges never see this, but it prevents catastrophe.

Move 6: Prepare for the accelerator interview

Even teams that don't win the top prize often get invited to the Colosseum accelerator. Prepare answers to the private accelerator application questions (already drafted in `HACKATHON_SUBMISSION.md`) with the same care as the public submission.

---

14. Implementation plan — timeline, milestones, budget

5-week timeline (April 10 → May 11)

Week 1 — Foundation + critical spike (Apr 10–17)

Goal: Prove the hardest technical unknowns work before touching UI.
- Monorepo scaffolded: Next.js 15, TypeScript, Tailwind, shadcn/ui, Supabase client, wallet adapter
- Solana Pay Transaction Request endpoint returns valid unsigned tx on devnet
- Jupiter v6 quote + swap-instructions working
- **Critical spike:** compose Jupiter swap + USDC transfer into one atomic transaction, fit in 1232 bytes (use ALTs if needed), confirm on devnet
- Mobile wallet scans a QR → signs → merchant devnet wallet receives USDC ✓

Week 2 — Core payment flow (Apr 17–24)

Goal: Full buyer payment flow end-to-end with error handling.
- Invoice creation API + UI
- Transaction Request full implementation (dynamic, based on buyer's input token)
- Error handling: no route, slippage exceeded, expired invoice, insufficient balance
- Both devnet and mainnet config paths
- Same flow works with 3+ different input tokens

Week 3 — Merchant dashboard + polish (Apr 24 – May 1)

Goal: A real product, not a hackathon prototype.
- Landing page (marketing)
- Merchant onboarding flow
- Dashboard: overview cards, transaction list, real-time updates (Supabase Realtime)
- Empty states, error states, loading states
- Mobile-responsive

Week 4 — Polish, security, mainnet dry-run (May 1–8)

Goal: Ship-quality.
- Line-by-line security review of every API route
- Supabase RLS policy audit with negative tests
- Accessibility pass (Lighthouse ≥95)

- i18n (English + Bahasa) extracted and wired
- JSDoc pass
- Bug bash across wallets, devices, network conditions
- **One real mainnet payment** (small amount, documented, gated behind a feature flag)

Week 5 — Submission package (May 8–11)

Goal: Make the judges' job easy.
- Demo video (≤3 min) recorded on deployed build
- Pitch video (≤2 min) recorded
- README finalized with 15-minute setup guide
- Landing page live
- Colosseum form submitted by May 10 (1-day buffer)
- Repo tagged v1.0.0

Full detailed milestone tracking is in `PLAN.md`.

---

15. Budget breakdown

OnPay is bootstrapped by a solo founder. Here's the full cost breakdown.

One-time / hackathon phase (April 10 – May 11)

Item	Cost (USD)	Notes
Domain name (onpay.app or onpay.io)	$20	One-year registration
Vercel Hobby plan	$0	Free tier is sufficient for the hackathon
Supabase Free tier	$0	Within limits for hackathon load
Helius RPC (developer tier)	$0	Free tier: 1M credits/month, enough for dev + demo
Solana devnet SOL	$0	Free airdrops
Mainnet test SOL	$10	For the one real mainnet dry-run payment
Test USDC (small amount for dry-run)	$10	To test real swap and settlement
Claude Pro / Max subscription	$0	Already a subscriber for other projects
Video editing software	$0	DaVinci Resolve (free)
Microphone / recording setup	$0	Existing equipment
Hackathon phase total	**$40**	

Recurring monthly (post-launch, operational)

Item	Cost (USD/month)	Notes
Vercel Pro	$20	Upgrade from Hobby when traffic grows; includes Vercel KV for rate limiting
Supabase Pro	$25	Once free tier is outgrown (~10k merchants)
Helius RPC Business	$99	Production traffic, priority sends
Domain renewal	$2	Amortized from yearly renewal
Monitoring (Sentry / Logflare)	$0	Free tiers
Email (transactional)	$0	Resend free tier
Monthly operational total	**~$146/month**	Conservative estimate

Growth phase (Months 3–12, if self-funded)

Item	Cost (USD)	Notes
Merchant onboarding kits (QR stickers, stands, signage)	$300	100 kits × $3
Travel for merchant visits (scooter fuel, coffee meetings)	$100/month	Bali-local
Video / content production	$50/month	Simple YouTube tutorials
Legal review for accelerator application + PSP partnership negotiation	$500 one-time	Local Indonesian counsel
PSP partnership setup (if needed)	$0–$2,000	Depends on partner terms
Growth phase total (6 months)	**~$1,500 + recurring**	

If Colosseum accelerator is granted

Colosseum accelerator cohorts typically offer $250k in funding. If OnPay is accepted:
- 60% ($150k) → founder salary + first hire (operator for Bali rollout)
- 25% ($62.5k) → growth (physical rollout, SDK development, PSP partnership)
- 15% ($37.5k) → infrastructure + legal + buffer

Break-even analysis (without external funding)

- **Monthly burn (bootstrapped):** ~$146 infrastructure
- **Merchant revenue at 0.3% fee:** ~$30/merchant/month average
- **Merchants needed to break even:** ~5 active paying merchants
- **Target to reach break-even:** Month 2 post-launch

The project can be fully self-sustaining after acquiring just a handful of active merchants. This is the power of non-custodial infrastructure: low fixed costs, high gross margin.

---

16. The competition & why we're different

Competitor	What they do	What they get wrong
Helio / MoonPay	Custodial crypto payment gateway	Takes custody → reintroduces middleman fees and risk that crypto should eliminate
Solana Pay (native reference impls)	Payment QR standard on Solana	Only supports same-token transfers — buyer must already hold the merchant's chosen token
Sphere Labs	B2B payments API on Solana	B2B-focused, not merchant-first UX; developers only, not café owners
Stripe Crypto	Custodial crypto checkout	High fees, custodial, still a walled garden with KYC
Coinbase Commerce	Custodial multi-chain checkout	Custodial, multi-chain UX is fragmented, not designed for in-person retail
Crossmint / Phantom Pay	NFT/digital-goods checkout	Wrong use case — designed for one-off NFT drops, not a $3 coffee

OnPay's gap: non-custodial + token-agnostic + QR-native + merchant-first + in one atomic transaction. Nobody else hits all five at once.

🔧 17. Technical architecture

This is the deep-dive section for developers.

System overview

OnPay consists of four components:

1. **Frontend (Next.js web app)** — merchant dashboard, invoice creation UI, public landing page.
2. **Backend (Next.js API routes)** — Solana Pay Transaction Request endpoints, Jupiter integration, invoice management, real-time status.
3. **Database (Supabase Postgres)** — merchants, invoices, payments, with Row-Level Security for merchant isolation.
4. **On-chain (Solana)** — Jupiter program, SPL Token program, and optionally a custom Anchor program for receipts/fees.

Component diagram

```
┌──────────────────────────────────────────────────────────────┐
│                        Next.js 15 App                        │
│                                                              │
│  ┌──────────────────┐         ┌──────────────────────────┐   │
│  │   Frontend (RSC)  │         │      API Routes          │   │
│  │                   │         │                          │   │
│  │  - Landing page   │         │  GET  /api/tx/[ref]      │   │
│  │  - Merchant auth  │         │  POST /api/tx/[ref]      │   │
│  │  - Dashboard      │◄────────│  POST /api/invoices       │   │
│  │  - Invoice form   │         │  GET  /api/invoices/[id]  │   │
│  │  - QR display     │         │  POST /api/merchants      │   │
│  │  - Tx history     │         │                          │   │
│  └──────────┬────────┘         └──────────┬───────────────┘   │
│           │                            │                    │
└──────────┼────────────────────────────┼────────────────────┘
           │                            │
           ▼                            ▼
  ┌──────────────────┐         ┌──────────────────────────┐
  │  Wallet Adapter   │         │     Supabase             │
  │  (Phantom, etc.)  │         │  - merchants table       │
  └──────────────────┘         │  - invoices table        │
                               │  - payments table        │
                               │  - RLS policies          │
                               │  - Realtime subscriptions │
                               └──────────┬──────────────┘
                                          │
                                          │ tx monitoring
                                          ▼
                               ┌──────────────────────────┐
                               │   Helius RPC + webhooks  │
                               └──────────┬──────────────┘
                                          │
                                          ▼
                               ┌──────────────────────────┐
                               │   Solana mainnet-beta    │
                               │                          │
                               │  - Jupiter program       │
                               │  - SPL Token program     │
                               │  - System program        │
                               │  - (opt) OnPay Anchor    │
                               └──────────────────────────┘
```

Data model

`merchants` table

```
create table merchants (
  id uuid primary key default gen_random_uuid(),
  wallet_address text unique not null,  -- base58 pubkey
  business_name text,
  settlement_token_mint text not null default 'EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v', --
USDC
  preferred_language text default 'en',
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- RLS: a merchant can only read/update their own row, authenticated by signed message
create policy "merchants_select_own" on merchants
  for select using (wallet_address = auth.jwt() ->> 'wallet_address');
```

`invoices` table

```
create table invoices (
  id uuid primary key default gen_random_uuid(),
  merchant_id uuid not null references merchants(id) on delete cascade,
  reference text unique not null,  -- 32 bytes base58
  amount_usd numeric(18, 6) not null,
  label text,
  memo text,
  status text not null default 'pending', -- pending | paid | expired | failed
  expires_at timestamptz not null default (now() + interval '10 minutes'),
  created_at timestamptz default now()
);

create index idx_invoices_reference on invoices(reference);
create index idx_invoices_merchant_status on invoices(merchant_id, status, created_at desc);
```

`payments` table

```
create table payments (
  id uuid primary key default gen_random_uuid(),
  invoice_id uuid not null references invoices(id),
  buyer_wallet text not null,
  input_mint text not null,         -- what the buyer paid with
  input_amount numeric(20, 0) not null,  -- raw token amount (no decimals)
  output_amount numeric(20, 0) not null, -- raw USDC amount received by merchant
  tx_hash text unique not null,
  confirmed_at timestamptz default now()
);
```

API surface

`POST /api/merchants`

Upsert a merchant profile after wallet connect. Authenticated via wallet signature verification.

```
// Request
{
  walletAddress: string,        // base58
  signature: string,            // signed message "OnPay login <nonce>"
  message: string,              // the signed message
  businessName?: string,
  preferredLanguage?: 'en' | 'id'
}

// Response
{
  id: string,
  walletAddress: string,
  businessName: string,
  settlementTokenMint: string,
}
```

`POST /api/invoices`

Create an invoice. Authenticated via wallet signature.

```
// Request
{
  amountUsd: number,
  label?: string,
  memo?: string,
}

// Response
{
  id: string,
  reference: string,
  amountUsd: number,
  expiresAt: string,
  paymentUrl: string,  // solana:... URL for QR encoding
}
```

`GET /api/tx/[reference]`

Solana Pay Transaction Request metadata. Called by the wallet after scanning.

```
// Response
{
  label: string,   // e.g., "Kopi Canggu"
  icon: string,    // absolute URL to merchant logo
}
```

`POST /api/tx/[reference]`

Solana Pay Transaction Request transaction builder. The wallet POSTs the buyer's account; we respond with a serialized unsigned transaction.

```
// Request (per Solana Pay spec)
{
  account: string,  // buyer wallet pubkey
  inputMint?: string,  // optional; if omitted, wallet picks via UI
}

// Response
{
  transaction: string,  // base64-encoded VersionedTransaction
  message: string,      // human-readable summary
}
```

This is the heart of OnPay. The implementation:

```
// /api/tx/[reference]/route.ts
export async function POST(req: NextRequest, { params }: { params: { reference: string } }) {
  // 1. Load invoice by reference
  const invoice = await db.invoices.findByReference(params.reference);
  if (!invoice) return new Response('Not found', { status: 404 });
  if (invoice.status !== 'pending') return new Response('Already paid or expired', { status: 409
});
  if (new Date(invoice.expiresAt) < new Date()) {
    await db.invoices.markExpired(invoice.id);
    return new Response('Expired', { status: 410 });
  }

  // 2. Parse buyer
  const { account, inputMint = WSOL_MINT } = await req.json();
  const buyer = new PublicKey(account);
  const merchant = await db.merchants.findById(invoice.merchantId);

  // 3. Compute USDC amount from USD price using Pyth or fixed 1:1 (USDC is pegged)
  const usdcAmount = BigInt(Math.round(invoice.amountUsd * 1_000_000)); // USDC has 6 decimals

  // 4. Jupiter quote: how much `inputMint` do we need to swap to get exactly `usdcAmount` USDC?
  const quote = await jupiter.quote({
    inputMint,
    outputMint: USDC_MINT,
    amount: usdcAmount,
    swapMode: 'ExactOut',
    slippageBps: 100, // 1% max
    onlyDirectRoutes: false,
  });

  // 5. Fetch swap instructions (not a pre-built tx)
  const swapIx = await jupiter.swapInstructions({
    userPublicKey: buyer,
    quoteResponse: quote,
    wrapAndUnwrapSol: true,
  });

  // 6. Get or create merchant's USDC associated token account
  const merchantUsdcAta = getAssociatedTokenAddressSync(
    USDC_MINT,
    new PublicKey(merchant.walletAddress)
  );

  // 7. Compose the transaction
  const instructions: TransactionInstruction[] = [
    // Set compute budget
    ComputeBudgetProgram.setComputeUnitLimit({ units: 600_000 }),
    ComputeBudgetProgram.setComputeUnitPrice({ microLamports: 10_000 }),

    // (Optional) Create merchant's USDC ATA if missing, funded by buyer
    createAssociatedTokenAccountIdempotentInstruction(
      buyer, merchantUsdcAta, new PublicKey(merchant.walletAddress), USDC_MINT
    ),

    // Jupiter swap setup + swap + cleanup instructions
    ...swapIx.setupInstructions.map(deserializeIx),
    deserializeIx(swapIx.swapInstruction),
    ...swapIx.cleanupInstructions.map(deserializeIx),
```

```
    // Transfer the swapped USDC from buyer's USDC ATA to merchant's USDC ATA
    createTransferCheckedInstruction(
      getAssociatedTokenAddressSync(USDC_MINT, buyer),
      USDC_MINT,
      merchantUsdcAta,
      buyer,
      usdcAmount,
      6
    ),

    // Include the reference key as a memo for on-chain indexing
    new TransactionInstruction({
      programId: MEMO_PROGRAM_ID,
      keys: [{ pubkey: new PublicKey(invoice.reference), isSigner: false, isWritable: false }],
      data: Buffer.from(invoice.reference, 'utf-8'),
    }),
  ];

  // 8. Build versioned tx with Address Lookup Tables (to fit in 1232 bytes)
  const { blockhash } = await connection.getLatestBlockhash('finalized');
  const messageV0 = new TransactionMessage({
    payerKey: buyer,
    recentBlockhash: blockhash,
    instructions,
  }).compileToV0Message([jupiterLookupTable]);

  const tx = new VersionedTransaction(messageV0);

  // 9. Serialize and return
  return Response.json({
    transaction: Buffer.from(tx.serialize()).toString('base64'),
    message: `Pay ${formatAmount(usdcAmount, 6)} USDC to ${merchant.businessName}`,
  });
}
```

Payment confirmation flow

Two options for detecting completed payments:

Option A — Polling (simpler, default MVP):
- After invoice creation, the merchant dashboard polls `GET /api/invoices/[id]/status` every 2 seconds until status = 'paid' or expired.
- A background worker in the API also polls the Solana RPC for transactions containing the reference key via `getSignaturesForAddress` on a hardcoded memo or the reference pubkey.

Option B — Helius webhook (better, production):
- Subscribe a Helius webhook to any transaction mentioning the reference pubkey.
- Helius POSTs to `/api/webhooks/helius` when detected.
- Handler updates `payments` table and emits Supabase Realtime event.
- Dashboard subscribes to Realtime and updates instantly.

MVP uses Option A; Option B is the Week 4 upgrade.

Anchor program (optional)

OnPay can optionally deploy a small Anchor program with two features:

1. **Payment receipt event** — emits a program event containing the invoice reference + amount + merchant, making indexing trivial.
2. **Fee collection PDA** — a program-derived account that receives the protocol fee as part of the atomic transaction. Non-custodial because the fee is collected inside the buyer-signed transaction, not held by OnPay off-chain.

Decision for MVP: skip the Anchor program. Pure Jupiter + SPL composition already demonstrates composability and works without custom on-chain code. Add the Anchor program only if judges' "Solana composability" score benefits outweigh the added surface area.

Transaction size budgeting

Solana's transaction size limit is 1232 bytes. A Jupiter swap + ATA creation + transfer + memo can easily hit 1400+ bytes without optimization. Mitigations:

- **Versioned transactions (v0).** Required for using Address Lookup Tables.
- **Address Lookup Tables (ALTs).** Jupiter provides ALTs; we use them to reduce account key bytes.
- **Skip ATA creation** if merchant already has a USDC ATA (cache this check).
- **Use** `createAssociatedTokenAccountIdempotent` so we can always include it without checking first.
- **Drop the memo** for on-chain indexing if we use Helius webhooks — the reference can be tracked via transaction parsing.

RPC strategy

- **Development:** default `https://api.devnet.solana.com`.
- **Production:** Helius Business plan. Provides priority sends (important during mainnet congestion), staked connection, and webhook subscriptions.
- **Fallback:** second RPC configured via env var; client retries on error.

🔧 18. Technology stack — what we're building with

Frontend

- **Next.js 15** (App Router, Server Components, React 19)
- **TypeScript** (strict mode, no `any`)
- **Tailwind CSS** + **shadcn/ui** (Radix primitives for accessibility)
- **lucide-react** (icons)
- **next-intl** (i18n for English + Bahasa)
- **qrcode.react** (QR rendering)

Solana integration

- `@solana/web3.js` — core client library
- `@solana/spl-token` — SPL token program bindings
- `@solana/pay` — Solana Pay URL builder + parser
- `@solana/wallet-adapter-react` + `@solana/wallet-adapter-react-ui` — wallet connection
- `@solana/wallet-adapter-phantom`, `-backpack`, `-solflare` — individual wallet adapters
- `@jup-ag/api` — Jupiter v6 API client (or direct HTTPS)

Backend / Data

- **Supabase** — Postgres + Realtime + RLS + optional auth
- `@supabase/supabase-js` — client
- **Upstash Redis / Vercel KV** — rate limiting and short-lived cache
- **Zod** — runtime input validation at API boundaries

Deployment & DevOps

- **Vercel** — frontend + API routes
- **GitHub Actions** — CI: typecheck, lint, test
- **Dependabot** — dependency hygiene
- **Sentry** (free tier) — error tracking

Testing

- **Vitest** — unit tests (transaction composition logic, utility functions)
- **Playwright** — end-to-end browser tests
- **Anchor test framework** — if/when a custom program is added

Code quality

- **ESLint** — with `@typescript-eslint` strict rules
- **Prettier** — code formatting
- **Husky** + **lint-staged** — pre-commit hooks
- **Commitlint** — conventional commit enforcement

AI tooling (disclosed per hackathon rules)

- **Claude Code** (Anthropic) — primary coding assistant
- **v0.dev** — UI component drafting
- **Colosseum Copilot** — hackathon-specific research

🔧 19. Smart contract and transaction design

Why no custom program for MVP

The core OnPay flow is composed entirely of existing Solana primitives:
- Jupiter's aggregator program (swap)
- SPL Token program (transfer)
- Memo program (reference tagging)

This means **we don't need to deploy and audit a custom program to ship a working product**. That's a feature, not a bug — less code = less attack surface = faster to ship and verify.

Judges score "composability with Solana primitives" favorably when you use existing programs well, not when you reinvent them.

When the custom Anchor program makes sense

We'll add a custom Anchor program in Phase 2 for:
1. **Protocol fee collection** — collect a 0.2–0.5% skim to a fee PDA as part of the atomic transaction.
2. **Subscription payments** — an approved-spender model where merchants can debit a subscribed wallet within configured limits.
3. **Refunds** — signed refund instructions that the merchant can execute from the dashboard.

Each of these requires program logic that doesn't fit into composing existing primitives alone.

Transaction anatomy (MVP, no custom program)

A single `VersionedTransaction` contains:

```
Compute Budget (2 instructions)
├── SetComputeUnitLimit(600_000)
└── SetComputeUnitPrice(10_000 µLamports)

ATA Setup (0-1 instructions, idempotent)
└── CreateAssociatedTokenAccountIdempotent(merchant_usdc_ata)

Jupiter Setup (0-N instructions)
├── CreateAssociatedTokenAccountIdempotent(buyer_intermediate_ata)
├── SyncNative (if wrapping SOL)
└── ... (other setup as Jupiter returns)

Jupiter Swap (1 instruction)
└── Route(input_mint → usdc, amount, slippage)

Jupiter Cleanup (0-N instructions)
└── CloseAccount(buyer_intermediate_ata, if any)

Final Transfer (1 instruction)
└── TransferChecked(buyer_usdc_ata → merchant_usdc_ata, amount, decimals=6)

Memo (1 instruction)
└── Memo(invoice_reference)
```

All inside a v0 transaction using Jupiter's lookup table. Total size target: <1232 bytes.

Atomic guarantee

This is the critical property: **either all instructions in the transaction succeed, or none of them do**. Solana's runtime enforces this at the protocol level. There is no intermediate state where the buyer has paid but the merchant hasn't received. There is no way for OnPay to "hold" funds even for a microsecond.

This is what we mean by "non-custodial." It's a mathematical property of the transaction, not a promise we make.

🔧 20. Security, non-custodial guarantees, risk mitigation

Non-custodial guarantee — auditable

OnPay's non-custodial claim is verifiable by reading the code. The guarantee holds as long as:

1. The transaction returned by `/api/tx/[reference]` is built such that the final SPL transfer destination is the merchant's publicly-known wallet address (stored in the `merchants` table, which the merchant controls via Supabase RLS).
2. No instruction in the transaction causes funds to be temporarily routed through any OnPay-controlled account.
3. The buyer's wallet — not OnPay — holds the private key that signs the transaction.

All three conditions are enforced in `/api/tx/[reference]/route.ts` and can be re-verified by anyone reading the source. There is no off-chain signing, no escrow, no hot wallet, no cold wallet held by OnPay.

Security review checklist (Week 4)

- [ ] **Input validation** at every API boundary using Zod
- [ ] **SQL injection impossible** — Supabase uses parameterized queries
- [ ] **No secrets in client bundles** — all RPC keys, Supabase service keys are server-only
- [ ] **Supabase Row-Level Security policies** verified with negative tests (merchant A cannot read merchant B's data)
- [ ] **Rate limiting** on all mutating endpoints (per-IP + per-wallet)
- [ ] **CSP headers** configured in `next.config.ts`
- [ ] **HTTPS only** enforced at Vercel level
- [ ] **Dependency audit** — `npm audit` clean, Dependabot enabled
- [ ] **Pinned dependency versions** — no `^` wildcards
- [ ] **Slippage cap** — hard max of 2%, default 1%
- [ ] **Reference key entropy** — 32 bytes crypto-random, not incrementing
- [ ] **Replay protection** — each invoice's reference is unique; paid invoices reject new payments
- [ ] **Wallet signature verification** for merchant-auth endpoints
- [ ] **Idempotent payment processing** — webhook handler survives duplicate deliveries
- [ ] **No custodial code paths** — reviewed line-by-line

Known risks and mitigations

Risk	Mitigation
Jupiter slippage on low-liquidity input tokens	Hard slippage cap; block tokens with no route; show slippage to buyer before signing
Transaction size > 1232 bytes	Use v0 tx + Jupiter's ALTs; test with worst-case routes
RPC congestion during mainnet demo	Helius priority sends + fallback RPC
Supabase downtime	Cached merchant data on frontend; invoice creation queued with retry
Jupiter API downtime	Fall back to direct DEX routing (Raydium, Orca) as emergency path
Regulatory exposure in Indonesia	Non-custodial design avoids most licensing; engage local counsel before fiat off-ramp
Frontend XSS	CSP + Next.js output escaping + no `dangerouslySetInnerHTML`
Merchant wallet key compromise	Out of OnPay's scope — but we document wallet security best practices for merchants

🔧 21. Development workflow and quality standards

Git workflow

- `main` branch is always deployable
- Feature work in feature branches: `feat/merchant-dashboard`, `fix/slippage-cap`, etc.
- Conventional commits: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`
- PRs even when solo — forces self-review
- Squash-merge to keep `main` history clean

Code standards

- **No `any` types.** Use `unknown` + narrowing.
- **JSDoc on every exported function** — at minimum a one-line description.
- **Explicit return types** on exported functions.
- **Imports sorted** and grouped (builtin → external → internal → relative).
- **No commented-out code** committed to main.
- **Error messages are user-facing** — translatable via i18n, never technical jargon.
- **Logs are structured** — JSON in production, pretty in development.

Testing

- **Unit tests** for pure logic: transaction composition, amount formatting, reference generation, slippage calculation.
- **Integration tests** for API routes: happy path + expected errors.
- **End-to-end tests** with Playwright: full merchant onboarding, invoice creation, (simulated) payment completion.
- **Manual testing checklist** for wallet compatibility across Phantom, Backpack, Solflare on iOS + Android.

CI pipeline (GitHub Actions)

On every push and PR:
1. Install dependencies (pnpm or npm, frozen lockfile)

2. Typecheck (`tsc --noEmit`)
3. Lint (`eslint .`)
4. Unit tests (`vitest run`)
5. Build (`next build`)
6. On PR: deploy preview to Vercel

On merge to `main`:
1. All of the above
2. Deploy to production Vercel
3. Run end-to-end Playwright tests against the preview

---

22. Risk analysis

Product risks

Risk	Likelihood	Impact	Mitigation
Merchants don't want to deal with crypto at all	Medium	Critical	Bali GTM specifically targets tourism merchants with crypto-holding customers; validate before scaling
Buyers find the flow too complex	Low	High	Ruthless UX testing; one-tap approval; clear preview before signing
Jupiter routes are too slippage-prone	Medium	Medium	Hard cap; surface slippage to buyer; block unroutable tokens
Non-crypto buyers can't use it	High	Low (for MVP)	Accepted trade-off for MVP; Phase 2 on-ramp solves this

Technical risks

Risk	Likelihood	Impact	Mitigation
Transaction composition exceeds size limit	Medium	High	Week 1 spike; use v0 + ALTs
Mainnet RPC rate limits during demo	Medium	High	Helius paid tier for demo period
Wallet adapter regressions	Low	Medium	Pin versions; regression test before demo
Supabase downtime	Low	Medium	Non-critical — merchants can still receive payments on-chain, just dashboard lags

Business risks

Risk	Likelihood	Impact	Mitigation
Regulatory crackdown on crypto payments in Indonesia	Medium	High	Non-custodial design sidesteps licensing; engage local counsel before fiat off-ramp
Local card processors match fees	Low	Medium	Even at same fees, OnPay wins on settlement speed and chargeback protection
Solana ecosystem slowdown	Low	Critical	Bet on Solana's fundamentals; monitor and migrate to another L1 only if forced
Merchant adoption slower than expected	High	Medium	Physical founder-led sales in Bali is the highest-leverage activity; be patient
Hackathon loss	Medium	Low	Win or lose, the product is built. OnPay exists post-hackathon regardless.

Personal / execution risks

Risk	Likelihood	Impact	Mitigation
Solo founder burnout	Medium	Critical	Weekly retrospectives; hard stop at 10pm; explicit rest days
Feature creep eats the deadline	High	High	Scope locked at MVP in `PLAN.md`; explicit non-goals list
Spike failure in Week 1	Medium	Medium	Degradation path documented (two-step payment fallback)

23. Success metrics

Hackathon success (by May 11, 2026)

- [ ] Public GitHub repo with clean README and 15-minute setup guide
- [ ] MVP deployed and functional on mainnet-beta
- [ ] At least 3 different input tokens tested end-to-end (SOL, BONK, USDT → USDC)
- [ ] Payment flow completes in <10 seconds wall-clock
- [ ] Lighthouse accessibility ≥ 95
- [ ] Zero `any` types in the codebase
- [ ] 100% of security review checklist completed
- [ ] Demo video ≤3 min, recorded on deployed build
- [ ] Pitch video ≤2 min
- [ ] Colosseum submission confirmed before May 10 (1-day buffer)

Accelerator success (if accepted)

- [ ] Onboard first 10 merchants in Bali within 30 days of accelerator start
- [ ] Process first $10k in real mainnet volume within 60 days
- [ ] Sign LOI with Indonesian PSP partner for fiat off-ramp within 90 days
- [ ] Publish `@onpay/sdk` v1.0 to npm

12-month success

- [ ] 100+ active merchants
- [ ] $100k+ monthly processed volume
- [ ] Shopify plugin in review
- [ ] First institutional (VC or strategic) conversation
- [ ] Break-even on operational costs

24. What happens after the hackathon

Win scenario

- Accept Colosseum accelerator offer (if extended)
- Hire one part-time operator for Bali rollout
- Ship Phase 2 roadmap: IDR off-ramp, Shopify plugin, SDK v1.0
- Target: 100 merchants + $100k/mo volume by end of 2026
- Consider seed round conversations if growth compounds

Lose-but-build scenario

- OnPay still exists. The codebase is open source, deployed, and working on mainnet.
- Do the Bali rollout anyway — the merchant pain is real regardless of hackathon outcome.
- Self-fund for 6 months from day-job income; hit break-even around Month 2.
- Continue shipping Phase 2 features incrementally.
- Reapply to future Solana hackathons with the updated product.

Pivot scenario (unlikely, documented for rigor)

If after 3 months of Bali rollout, merchant adoption is under 20 paying merchants:
- Interview every declining merchant to understand why
- Consider pivot to B2B (developer-first SDK for existing e-commerce stores) where merchant onboarding friction is replaced by developer integration friction — and developers are easier to reach
- Consider geographic pivot — maybe Bali isn't the right beachhead

25. Closing thoughts

OnPay is not a new payment network. It's an orchestration layer over existing Solana primitives — Solana Pay, Jupiter, SPL Token — glued together into a product a non-technical café owner can use in 60 seconds.

The technical work is real (transaction composition, UX, security) but it's not heroic. The hard part is not the code. The hard part is picking the right architecture (non-custodial, any-to-stable, one-tap) and then shipping it before the custodial incumbents catch up.

The winning move is speed + focus + distribution. Ship the MVP. Win (or place in) the hackathon. Walk into Bali cafés. Solve a real problem for real people.

Everything else follows.

Document version 1.0 — prepared for Solana Frontier Hackathon 2026 submission. Living document — will be updated as the project evolves. For the latest version, see the repository at `github.com/me-workspace/onpay_solana`.

OnPay · Flexible Input, Stable Output · Built in Bali